DFSha: Informe técnico

Curso: ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos, 2026-2
Proyecto 1: Sistema de archivos distribuido con alta disponibilidad, rendimiento y seguridad
Integrantes: Camilo Ruiz · Carlos Ochoa · Thomas Osorio
Repositorio: https://github.com/thomaszambrano/DFS-Topicos-Telematica
Video demostración: https://youtu.be/No2hkhglwMw
Fecha: 25 de octubre de 2026

---

1. Objetivos y marco teórico

1.1 Objetivo general

Diseñar e implementar un DFS que almacene archivos grandes repartidos entre varios nodos. La escritura y la lectura deben ser distribuidas, y el sistema debe considerar alta disponibilidad, rendimiento, transparencia y seguridad.

1.2 Objetivos específicos

1. Partir cada archivo en chunks y repartir sus chunks entre varios DataNodes (RNF4).
2. Replicar cada chunk en R nodos distintos y reparar las réplicas perdidas sin intervención humana (RNF2).
3. Garantizar que un lector nunca vea una versión a medias de un archivo (RNF3).
4. Separar el plano de control (metadatos) del plano de datos (bytes), para que el coordinador no sea el cuello de botella (RNF1, RNF5).
5. Descubrir dinámicamente la ubicación de los datos: el cliente solo conoce al ControlNode (RNF7).
6. Autenticar usuarios y aislar los archivos de cada uno (RNF6).
7. Usar REST entre el cliente y el servidor, y gRPC entre los nodos, como exige el enunciado.

1.3 Alcance logrado

El sistema corresponde al Hito 3 parcial. La tabla de la sección 6.4 muestra el estado de cada requisito del enunciado.

1.4 Marco teórico

Sistema de archivos distribuido (DFS). Un DFS ofrece a sus clientes un único espacio de nombres de archivos, aunque los datos estén guardados en varias máquinas. Se busca *transparencia*: el usuario lee y escribe rutas como en un disco local, sin saber dónde están los bytes ni cuántas copias existen.

Arquitectura maestro/trabajadores (GFS, HDFS). El Google File System y su versión abierta, HDFS, separan dos funciones. Un nodo maestro (*NameNode*) guarda solo los metadatos: el namespace y la ubicación de cada bloque. Los nodos de datos (*DataNodes*) guardan los bloques. El cliente pide los metadatos al maestro y transfiere los bytes directamente con los nodos de datos, de modo que el maestro no sea el cuello de botella. DFSha sigue este modelo; su maestro es el ControlNode.

Particionamiento por bloques. Cada archivo se divide en bloques (*chunks*) de tamaño fijo, que se reparten entre varios nodos. Así un archivo puede ser más grande que cualquier disco, y su lectura y escritura se pueden hacer en paralelo contra varios nodos. Con bloques grandes (64 MB en GFS y DFSha) hay pocos metadatos por archivo.

Replicación y tolerancia a fallos. Cada bloque se guarda en R nodos distintos (R = 3 es el valor usual). Si un nodo falla, el bloque sigue disponible en las demás réplicas. El maestro detecta el fallo porque el nodo deja de enviar *heartbeats* periódicos, y entonces ordena copiar los bloques afectados a otros nodos hasta recuperar R réplicas (*re-replicación*). Los checksums de cada bloque permiten detectar réplicas corruptas.

Consistencia y el teorema CAP. Según el teorema CAP, ante una partición de red un sistema distribuido debe elegir entre consistencia y disponibilidad. Los DFS como HDFS usan semántica *write-once*: un archivo se publica completo y de forma atómica solo cuando todos sus bloques están escritos, así ningún lector ve una versión a medias. DFSha adopta esta semántica (D-005).

Comunicación: REST y RPC. REST expone recursos por HTTP con representaciones como JSON. Es sencillo y lo puede consumir cualquier cliente. RPC (aquí gRPC sobre HTTP/2 con Protocol Buffers) permite invocar procedimientos remotos con contratos tipados y flujos (*streams*) binarios. Por eso es adecuado para el tráfico interno entre nodos.

Balanceo de carga en la colocación. Para decidir en qué nodos poner cada bloque, DFSha usa la técnica de “las dos opciones” (*power of two choices*, Mitzenmacher): toma dos candidatos al azar y se queda con el de mejor puntuación (D-016). Con esto la carga queda casi tan equilibrada como si se revisaran todos los nodos, a un costo mucho menor.

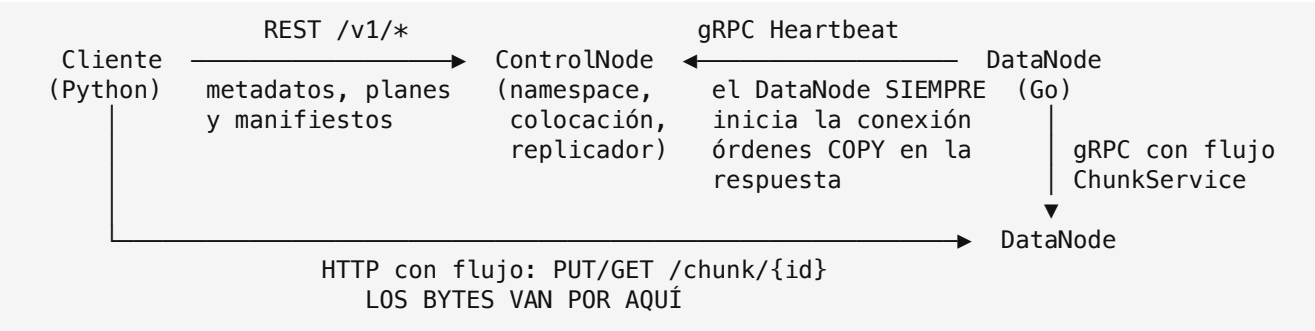
---

2. Arquitectura

2.1 Modelo elegido

Seguimos la **opción 1 del enunciado: cliente/servidor, con el servidor compuesto por varios servicios autónomos**. Es un DFS basado en objetos con particionamiento interno por bloques, al estilo de HDFS y GFS:

- **ControlNode** (Go): guarda el namespace jerárquico, decide dónde va cada chunk, mantiene la membresía de los DataNodes y coordina la re-replicación. **No transporta bytes de archivos.**
- **DataNodes** (Go, 4 instancias): guardan los chunks en disco, verifican su checksum, reportan su inventario y copian chunks entre ellos.
- **Cliente** (Python, solo biblioteca estándar): una consola tipo Linux (`ls`, `cd`, `mkdir`, `put`, `get`, …) y un SDK importable.



2.2 Enlaces de comunicación

Enlace	Protocolo	Contrato	Quién inicia
Cliente ↔ ControlNode	REST/JSON	`api/openapi.yaml`	Cliente
Cliente ↔ DataNode	HTTP con flujo	`PUT/GET /chunk/{id}`	Cliente
DataNode → ControlNode	gRPC unario `DataNodeService.Heartbeat`	`api/dfsha.proto`	Siempre el DataNode
DataNode ↔ DataNode	gRPC con flujo `ChunkService.ReplicateChunk`	`api/dfsha.proto`	El nodo origen
ControlNode ↔ ControlNode	(pendiente: Raft)	—	—

Entre los nodos se usa **gRPC, que es RPC tipado**: los mensajes se definen en protobuf y el código cliente y servidor se genera con `protoc`. Para el cliente se eligió REST porque es un contrato abierto, fácil de consumir desde cualquier lenguaje y documentable con OpenAPI.

2.3 Tres reglas que explican el diseño

1. **Los datos no pasan por el ControlNode.** Su tráfico crece con el *número* de chunks, no con su tamaño. Así un solo coordinador puede atender muchos clientes (RNF1).
2. **El DataNode siempre inicia la conexión.** El ControlNode nunca llama a un DataNode: las órdenes (por ejemplo `COPY_CHUNK`) viajan en la respuesta al heartbeat. Un nodo nuevo solo necesita la dirección del ControlNode, y puede estar detrás de NAT.
3. **Estado blando y estado duro.**
    - Las ubicaciones `chunk → nodos` no se guardan: se reconstruyen con los inventarios que llegan en cada heartbeat (estado blando, D-002).
    - El namespace solo existe en el ControlNode (estado duro). Por eso su persistencia con WAL es la prioridad pendiente.

2.4 Modelo de datos

- **Inode:** directorio u objeto, con dueño, permisos y un puntero `Current` a la versión vigente.
- **Version:** la lista ordenada de chunks, el tamaño y el sha256 del archivo completo. Es inmutable una vez publicada.
- **Chunk:** un identificador aleatorio, el índice, el tamaño y un checksum SHA-256. Mide 64 MB por omisión, y 4 MB en la demostración (D-004).
- **Upload:** el estado temporal de una carga en curso, con su lease, su clave de idempotencia y los chunks asignados (D-005).

3. Implementación

3.1 Escritura (`put`)

```
POST /v1/obj/open              → upload_id, chunk_size, lease sobre la ruta
  por cada chunk (en paralelo):
    POST /v1/obj/{id}/chunk    → chunk_id + R DataNodes destino (decide el ControlNode)
    PUT  http://{dest}/chunk   → bytes directo a cada destino; se anotan los ACK
POST /v1/obj/{id}/commit       → publica la versión (atómico), exige W ACK por chunk
```

- **Atomicidad (RNF3):** publicar una versión es cambiar el puntero `Inode.Current` bajo candado. Mientras el commit no termina, los lectores ven la versión anterior completa.
- **Concurrencia:** el lease sobre la ruta hace que una segunda carga simultánea reciba `409 Conflict`. El cliente nunca reintenta un 409.
- **Idempotencia:** la cabecera `Idempotency-Key` evita versiones duplicadas cuando el cliente reintenta tras un timeout.
- **Quórum (D-013):** con R=3 y W=2, una carga sobrevive a la caída de un destino. Solo se registran como réplicas los nodos que confirmaron.
- **Integridad:** el DataNode verifica el checksum *antes* de aceptar los bytes, y escribe por renombrado atómico (D-009), así que nunca queda un chunk a medias en disco.

3.2 Lectura (`get`)

1. `GET /v1/obj/manifest` devuelve la lista ordenada de chunks, con las réplicas **vivas** de cada uno.
2. El cliente descarga los chunks en paralelo (4 hilos) y escribe cada uno en su posición con `os.pwrite`. La memoria usada es `hilos × chunk_size`, no el tamaño del archivo.
3. Si una réplica falla, el cliente pasa a la siguiente sin esperar los reintentos.
4. Al final verifica el checksum de cada chunk y el sha256 del archivo completo.

3.3 Membresía y detección de fallos

- Cada DataNode envía un heartbeat cada 3 s con su capacidad, su ocupación y su inventario de chunks.
- Estados: `ALIVE`, luego `SUSPECT` (sin heartbeat por 9 s) y después `DEAD` (por 30 s). El estado intermedio evita que un reinicio rápido dispare copias innecesarias.

3.4 Re-replicación (D-014)

1. Cada 5 s el replicador busca los chunks de versiones vigentes que tienen menos de R réplicas vivas. Atiende primero los que tienen menos copias.
2. Para cada uno elige un origen vivo y un destino sin copia, ambos de menor carga, y deja una orden `COPY_CHUNK` en la cola del origen.
3. El origen recibe la orden en la respuesta a su próximo heartbeat y envía el chunk al destino por `ChunkService.ReplicateChunk`, un stream gRPC.
4. El destino verifica el checksum antes de aceptar la copia.

El factor R se cambia en caliente con `PUT /v1/config/replication`, sin reiniciar nada.

3.5 Heurística de colocación (D-016)

`internal/controlnode/placement.go`. Para cada réplica de un chunk:

1. Se filtran los nodos vivos que tienen espacio para **ese** chunk.
2. Se eligen **dos candidatos al azar** y se queda el de mayor puntaje: `score = 0.5·(1 − ocupación) + 0.3·(1 − carga en curso) + 0.2·dispersión`. La dispersión vale 0 si el nodo ya guardó el chunk anterior del archivo, lo que esparce el archivo entre los nodos.
3. El nodo elegido sale de la lista: dos réplicas nunca caen en el mismo nodo.
4. Se reserva su ocupación de forma provisional (`Registry.Reserve`), y el siguiente heartbeat la corrige.

Elegir siempre el mejor nodo causaría un efecto manada: como la ocupación solo se actualiza con cada heartbeat, todas las escrituras de esos segundos irían al mismo nodo. Tomar dos al azar reparte casi tan bien como revisar todos y evita ese problema.

3.6 Seguridad (D-017)

- **Registro:**
  - El nombre de usuario se valida con `^[a-z0-9_-]{1,32}$`.
  - La contraseña debe tener al menos 8 caracteres y se guarda con **Argon2id** (64 MB de memoria y salt aleatorio de 16 bytes por usuario).
  - Al registrarse, el usuario recibe su directorio `/home/<usuario>`.
- **Login:** entrega un token `base64(usuario|expira).HMAC-SHA256` que vence en 15 min.
  - El servidor lo verifica solo con su clave, sin guardar sesiones.
  - Las comparaciones se hacen en tiempo constante.
  - Un usuario inexistente y una contraseña incorrecta reciben el mismo error, para no revelar qué usuarios existen.
- **Middleware:** toda ruta exige `Authorization: Bearer`, salvo `/health`, los `GET` de diagnóstico y `/v1/auth/*`. Responde 401 si el token falta, es inválido o venció.
- **Aislamiento por usuario:**
  - Cada operación se limita al `/home` del solicitante (403 fuera de él).
  - Antes de comparar rutas se aplica `path.Clean`, así que `../` no sirve para salir.
  - Los listados de `/` y `/home` solo muestran el home propio.
- **Cliente:** pide la contraseña sin eco y renueva el token solo si vence durante la sesión.

3.7 Tecnologías

Componente	Tecnología
ControlNode y DataNode	Go 1.27, `net/http`, `google.golang.org/grpc`, `golang.org/x/crypto/argon2`
Contratos	Protocol Buffers v3 (`api/dfsha.proto`), OpenAPI 3.0.3 (`api/openapi.yaml`)
Cliente	Python 3, solo biblioteca estándar
Despliegue	Docker Compose: 1 ControlNode + 4 DataNodes con volúmenes propios

3.8 Estructura del código

```
api/                      contratos: openapi.yaml y dfsha.proto
cmd/controlnode, datanode ejecutables y configuración por variables de entorno
internal/controlnode/     namespace, registry, placement, allocator, replicator, auth, ser
internal/datanode/        store (disco), servidor HTTP, ChunkService gRPC, heartbeat
internal/types/           modelo de datos compartido
client/dfsha/             SDK (api.py), transferencia paralela, consola (shell.py)
scripts/verificar.sh      verificación de extremo a extremo
```

---

4. Pruebas

4.1 Pruebas unitarias e integración (`go test -race ./internal/...`)

Área	Pruebas	Qué demuestran
Almacenamiento del DataNode	`TestPutGetRoundTrip`, `TestPutRechazaChecksumIncorrectoSinDejarBasura`, `TestGetDetectaCorrupcionEnDisco`, `TestIDsPeligrososSeRechazan`, `TestPutRepetidoEsIdempotente`, …	Integridad por checksum, ausencia de chunks a medias, IDs seguros, idempotencia
Copia entre DataNodes	`TestCopyChunkBetweenNodes`, `TestCopyChunkRefusesCorruptSource`, `TestReplicateChunkPipeline`	Stream gRPC, rechazo de un origen corrupto, reenvío en cadena
Namespace	`TestMkdirYList`, `TestRemoveDirectorioNoVacio`, `TestRutasNoEscapanDeLaRaiz`, `TestRename`, …	Semántica de un sistema de archivos jerárquico
Replicador	`TestPlanReplicationRepairsUnderReplicated`, `TestPlanReplicationCountsPendingCopies`, `TestAllocatorRejectsWritesWithSingleNode`	Reparación sin duplicar órdenes y exigencia de quórum
Colocación	`TestSelectBalanceaOcupacion`, `TestSelectSinEspacioSuficiente`	Réplicas en nodos distintos, ocupación con menos de 10 % de diferencia en 100 chunks, rechazo cuando no hay espacio
Seguridad	`TestSinTokenEs401`, `TestContraseñaIncorrectaEs401`, `TestAislamientoPorUsuario`	401 sin token, con token falso o con contraseña incorrecta; 403 en un home ajeno, también con `../`; listados filtrados

Todas pasan con el detector de condiciones de carrera activado.

4.2 Verificación de extremo a extremo (`make verify`)

Sobre el clúster real en Docker:

1. Revisa la membresía: al menos 3 DataNodes vivos.
2. Crea un directorio en el home del usuario.
3. Comprueba que el namespace responde 401 sin token y que otro usuario recibe 403.
4. Sube un archivo aleatorio de 40 MB.
5. Lista los chunks **desde dentro de cada contenedor**, para comprobar que están físicamente en el volumen de cada nodo.
6. Descarga el archivo y compara su sha256 con el original.

4.3 Pruebas manuales de alta disponibilidad

1. Se sube un archivo de 80 MB con R=3.
2. Se apaga `dfsha-datanode2` con `docker stop`.
3. Se espera a que el ControlNode lo marque `DEAD` y ordene las copias de reparación (líneas `REPARAR` en el log).
4. Con el nodo caído, se descarga el archivo y se compara su sha256.
5. Se cambia R a 4 en caliente y se observa la creación de la cuarta réplica.

---

5. Resultados

Prueba	**Resultado observado**
`go test -race ./internal/...`	Todas pasan (`controlnode`, `datanode`)
`make verify`	Verde: 40 MB en 10 chunks × 3 réplicas = 30 copias en los 4 nodos; sha256 idéntico
Seguridad	401 sin token y al cambiar R sin login; 403 al listar el home de otro usuario
Balanceo	Tras subir 40 MB y luego 80 MB, los nodos quedaron con **21 / 23 / 23 / 23** chunks. El nodo que se unió tarde (y estaba más vacío) recibió 20 copias del segundo archivo, contra 13–14 de los demás: la heurística compensó el desbalance.
Caída de un nodo	El ControlNode emitió órdenes `REPARAR … réplicas 2/3`; la descarga siguió funcionando desde otras réplicas, con sha256 idéntico; las réplicas volvieron a 3 sin datanode2
R en caliente	`PUT /v1/config/replication` con R=4: el log muestra `REPARAR … réplicas 3/4` en el siguiente ciclo del replicador
Rendimiento (local)	Descarga de 40 MB a unos 225 MB/s con 4 hilos (en Docker, en una sola máquina; es indicativo, no un benchmark)

---

6. Discusión

6.1 Consistencia (RNF3)

- Una **única autoridad de metadatos** con commit atómico por intercambio de puntero da **consistencia lineal sobre el namespace**: un archivo se ve completo o no se ve.
- Las réplicas de un chunk nunca divergen, porque los chunks son inmutables: sobrescribir un archivo crea chunks nuevos y una versión nueva.
- La consistencia entre réplicas se reduce entonces a “existe o no existe”, y la resuelve el replicador.

6.2 Disponibilidad (RNF2)

- La caída de un DataNode no interrumpe ni las lecturas ni las escrituras, gracias a R=3 y W=2.
- La redundancia se restaura sola.
- El **ControlNode es un punto único de fallo**: si cae, el servicio se detiene. La solución prevista es replicar el ControlNode con Raft sobre un WAL.

6.3 Escalabilidad y rendimiento (RNF1, RNF5)

- El ControlNode solo procesa metadatos.
- El ancho de banda crece con el número de DataNodes, y lecturas y escrituras van en paralelo por chunk.
- La heurística evita los puntos calientes.
- Limitación: el cliente sube las R copias él mismo, así que el tráfico de subida es R veces el archivo. El reenvío en cadena entre DataNodes ya está implementado y probado, pero el cliente aún no lo usa (D-015).

6.4 Estado frente a los requisitos del enunciado

Requisito	Estado
RF1 Gestión del FS (`ls`, `cd`, `mkdir`, `rmdir`, `rm`, `mv`, `stat`)	✅
RF2 Transferencia (`put`/`get`, SDK `send`/`receive`)	✅
RF3 Acceso (`open`/`read`/`write`/`lock`)	⚠️ Parcial: modelo de objetos (escritura completa, lease como lock de escritura), sin lectura aleatoria
RNF1 Escalabilidad	✅ plano de datos separado y heurística; ⚠️ un solo ControlNode
RNF2 Alta disponibilidad	✅ réplicas, quórum, re-replicación; ❌ ControlNode replicado
RNF3 Consistencia	✅ commit atómico, chunks inmutables, leases
RNF4 Particionamiento	✅ chunks repartidos entre los 4 nodos
RNF5 Rendimiento y concurrencia	✅ transferencias paralelas, leases; ⚠️ sin pipeline
RNF6 Seguridad	⚠️ usuario/contraseña con Argon2id, token y aislamiento por usuario; ❌ TLS, cifrado en reposo, seguridad entre nodos
RNF7 Transparencia	✅ ubicación dinámica (el cliente solo conoce al ControlNode), consola tipo Linux
gRPC entre nodos	✅ `DataNodeService` y `ChunkService`

7. Limitaciones y trabajo futuro

1. **WAL y snapshot** del namespace y los usuarios. Hoy viven en memoria y se pierden al reiniciar el ControlNode.
2. **Replicación del ControlNode con Raft**, para eliminar el punto único de fallo.
3. **Pipeline de escritura**: que el cliente envíe al primer destino con los demás en `downstream`.
4. **Recolección de basura**:
   - borrar los chunks huérfanos de versiones reemplazadas;
   - borrar las copias sobrantes al bajar R (ya existe `DELETE_CHUNK` en el DataNode, pero el ControlNode aún no la emite).
5. **TLS** en REST y gRPC, **tokens de chunk** para que los DataNodes autoricen cada acceso, y **cifrado en reposo**.
6. **Rebalanceo activo** y drenado de nodos.

8. Decisiones de diseño

El registro completo, con alternativas descartadas y costos, está en `DECISIONES.md`:

ID	Decisión
D-002	Las réplicas son estado blando, reconstruido con los heartbeats
D-004	Chunk de 64 MB por omisión, 4 MB en la demostración
D-005	`Upload` separado de `Version`
D-009	Escritura atómica por renombrado en el DataNode
D-010	Ubicaciones esperadas de corta vida, para leer lo recién escrito
D-011	Namespace en memoria (limitación declarada)
D-013	El commit registra solo réplicas confirmadas y exige quórum W
D-014	Re-replicación por órdenes en la respuesta al heartbeat
D-015	El cliente escribe en los R destinos; el pipeline queda preparado
D-016	Colocación por dos candidatos aleatorios con puntuación
D-017	Argon2id y token HMAC autoverificable

9. Reproducción

Requisitos: Docker con Compose, Python 3 y Go 1.27 (este último solo para los tests).

```
git clone https://github.com/thomaszambrano/DFS-Topicos-Telematica
cd DFS-Topicos-Telematica
make up            # 1 ControlNode + 4 DataNodes
make verify        # verificación de extremo a extremo
make shell         # consola; crea el usuario la primera vez
go test -race ./internal/...
```

10. Referencias

- Ghemawat, Gobioff y Leung. *The Google File System*. SOSP 2003.
- Shvachko, Kuang, Radia y Chansler. *The Hadoop Distributed File System*. MSST 2010.
- Mitzenmacher. *The Power of Two Choices in Randomized Load Balancing*. IEEE TPDS, 2001.
- Biryukov, Dinu y Khovratovich. *Argon2* (RFC 9106), 2021.
- Ongaro y Ousterhout. *In Search of an Understandable Consensus Algorithm (Raft)*. USENIX ATC 2014.